

架构辩论赛记录

AI问答

问：
校园互助平台推荐什么架构

答：
搭建校园互助平台（如代取快递、跳蚤市场、学习资料共享等），架构的选择关键在于**高并发处理能力**（抢单场景）、**实时性**（聊天通知）以及**易维护性**。

建议采用**前后端分离的微服务架构**（中大型）或**云原生无服务器架构**（初创/学生项目）。

1. 推荐技术栈（主流组合）

维度	推荐方案	说明
前端/移动端	uni-app 或 微信小程序原生	校园场景下，小程序获客成本最低，无需下载。
后端框架	Spring Boot (Java) 或 Go-Zero (Go)	Java 生态稳，Go 适合高并发下的轻量化部署。
数据库	MySQL + Redis	MySQL 存储主体数据，Redis 处理高频缓存和分布式锁（抢单）。
实时通信	WebSocket 或 MQTT	用于即时聊天和订单状态变更通知。
文件存储	OSS / Cos	存储用户上传的图片、课件资料。

2. 逻辑架构设计

为了保证平台的可扩展性，建议将功能模块化：

用户与安全层

- 统一身份认证：**接入学校的 **CAS/OAuth2** 接口（如果学校开放），或者通过**学信网截图/学校邮箱**进行实名认证。
- 权限控制：**区分普通学生、志愿者、管理员。

业务服务层（微服务化建议）

- 订单/任务模块：**处理发布、接单、评价逻辑。使用 **Redis 队列** 防止超卖或多人同时抢一单。
- 社交/消息模块：**集成环信或自建 **WebSocket** 维持双方沟通。
- 积分/虚拟币系统：**用于互助激励，记录用户的“贡献值”。
- 内容审核服务：**利用 **AI** 接口（如阿里云内容安全）自动过滤违规图文。

数据处理层

- 搜索推荐：**采用 **Elasticsearch** 实现关键词搜索（如“求购考研英语资料”）。

* **地理位置:** 利用 MySQL 的 **Spatial Index** 或 Redis 的 **GeoHash** 实现“附近的任务”功能。

3. 针对“校园场景”的特殊设计

> **“抢单”瞬时并发峰值:**

> 校园平台在特定时间（如下课后、双11）会有瞬时流量。建议在架构中引入 **消息队列 (RabbitMQ/Kafka)**。当大量接单请求涌入时，先进入队列削峰填谷，避免后端数据库直接宕机。

> **部署建议:**

> 如果是学生创业或社团项目，强烈推荐使用 **微信小程序·云开发 (Tencent CloudBase)**。

> * **优势:** 无需购买服务器、域名和备案，自带 CDN 和数据库，运维成本几乎为零。

4. 架构示意图（简化版）

\$\$

\text{小程序端} \rightarrow \text{API Gateway (Nginx)} \rightarrow \begin{cases} \text{Auth Service (鉴权)} \\ \text{Order Service (订单)} \\ \text{Message Service (实时消息)} \end{cases} \rightarrow \text{MySQL / Redis}

\$\$

你是打算从零开始自研，还是基于现有的开源框架进行二次开发？

问：

基于问卷的需求整理结果

根据问卷调研结果，以下是校园互助服务平台的主要需求整理：

1. **用户注册与认证**

- 用户希望通过学号进行注册和认证，以确保平台用户均为校内学生，增加安全性。
- 用户对学号认证方式普遍接受，认为可以防止外校人员进入平台。
- （注：小部分用户有保护个人隐私、匿名或小号以提高自由度的需求）

2. **核心功能需求**

- 跑腿服务：包括快递代取、外卖代送、物品搬运等。
- 失物招领：方便用户发布和查找失物信息。
- 二手交易：支持学生之间的二手物品买卖。

- 学习互助：包括学习资料分享、课程辅导、疑问解答等。
- 生活帮忙：如咨询、求助、社区讨论等。
- 组队匹配：支持活动组队、课程组队、运动搭子等。

3. **交流与反馈**

- 用户希望在互助过程中能够与对方进行线上交流，以便更好地协调任务细节，部分用户希望平台同时有作为社交平台的需求。
- 用户希望平台能够提供评价反馈机制，以便了解互助双方的信誉和服务质量。
- 用户希望通过双方共同确认与平台作为中间人保留酬劳的方式确保任务完成与及时结算。

4. **酬劳与积分**

- 多数用户愿意接受积分作为酬劳，积分可用于兑换平台服务或商品。
- 部分用户希望可以直接获得现金或其他形式的酬劳。

5. **管理与维护**

- 用户希望平台能够管理言论、个人隐私、交易信誉等方面，确保平台氛围良好。
- 用户希望平台能够定期更新内容，保持活跃度，并提供便捷的搜索和推荐功能。

6. **收费方式**

- 用户对收费方式意见不一，部分用户希望完全免费，部分用户接受部分功能收费或自愿捐款。

--- 结合课堂所学的架构风格（MVC、分层、微服务、事件驱动等），分析至少 2 种候选架构方案，从以下维度进行对比：

对比维度	方案 A	方案 B
开发复杂度		
可维护性		

| 可扩展性 | | |

| 性能 | | |

| 团队学习成本 | | |

| 10 周内可行性 | | |

并比较优劣

答：

这份需求整理得非常扎实，涵盖了从身份核验到酬劳结算的核心闭环。针对校园互助平台这种**业务模块多（跑腿、交易、社交）、用户基数波动大、且需要在 10 周内交付**的特点，我为你选择了两种最具代表性的架构方案进行对比：

- * ****方案 A：分层架构（Layered Architecture / Monolithic）**** - 经典的“大一统”模式。
- * ****方案 B：微服务架构（Microservices Architecture）**** - 现代的“解耦”模式。

方案 A：分层架构（Layered Architecture）

这是最传统的架构模式，通常分为：****表现层（UI）、业务逻辑层（Service）、数据访问层（DAO）****。对于校园项目，所有功能（跑腿、失物、积分等）都运行在同一个进程中。

- * ****优势：**** 开发效率极高，调试方便，不需要处理复杂的分布式事务和网络延迟。
- * ****劣势：**** 代码耦合度高。如果“跑腿服务”的代码写出了 Bug 导致崩溃，整个平台（包括失物招领和积分）都会瘫痪。

方案 B：微服务架构（Microservices Architecture）

将不同功能拆分为独立的服务，例如：用户服务、任务服务（跑腿）、交易服务（二手）、消息服务（社交）。服务之间通过 REST API 或消息队列通讯。

- * ****优势：**** 模块化极强。后期如果“二手交易”流量激增，可以单独给它增加服务器资源，而不影响其他功能。
- * ****劣势：**** 基础设施搭建成本极高。你需要处理服务注册、发现、负载均衡以及分布式数据一致性（比如积分扣除成功但任务状态没更新怎么办）。

架构方案对比表

对比维度	方案 A：分层架构（Monolith）		方案 B：微服务架构（Microservices）	
:---	:---		:---	
开发复杂度	**低**：单一工程，逻辑直观。			**高**：涉及服务拆分、接口定义和分布

式通信。 |

| ****可维护性**** | ****中****: 初期容易, 后期代码量大后会变得臃肿。 | ****高****: 服务解耦, 单个模块的修改不影响全局。 |

| ****可扩展性**** | ****低****: 只能对整个应用进行水平扩展。 | ****高****: 可针对特定高频模块 (如跑腿) 精准扩容。 |

| ****性能**** | ****高****: 本地方法调用, 无网络开销。 | ****中****: 存在跨服务的网络延迟。 |

| ****团队学习成本**** | ****低****: 掌握基础 Web 开发即可上手。 | ****高****: 需学习 Docker, API 网关, RPC 等技术栈。 |

| ****10 周内可行性**** | ****极高****: 非常适合快速迭代并上线。 | ****低****: 光是搭建底层设施可能就要花掉 3-4 周。 |

> ****特别提示:**** 针对需求 3 中的“社交平台需求”和“实时确认”, 建议在架构中引入一个轻量级的 ****事件驱动 (Event-Driven)**** 逻辑, 例如使用 Redis 的发布/订阅功能来处理即时消息提醒, 这不需要重构整个架构, 但能极大提升用户体验。

问:

扮演架构师角色进行分析

答:

你好! 很高兴能以****架构师****的身份为你这份方案进行深度复盘。

首先, 我必须为你和团队点个赞。在 10 周的紧迫期限内, 能抵御住“微服务”这种大热概念的诱惑, 清醒地选择****分层架构 (Monolith)****, 这说明你们不仅懂技术, 更懂****工程权衡 (Trade-off)****。

在软件架构界有一句名言: **“**没有最好的架构, 只有最合适的架构。**”**针对你们的校园项目, 以下是我的专业分析与进阶建议。

🏠 方案确认: 为什么“模块化单体”是明智之举

你们选择方案 A 并采用“模块化单体”的思路, 完美规避了分布式系统的“三大坑”: 网络延迟、分布式事务处理、以及运维自动化成本。

1. 应对 10 周交付的“生存法则”

微服务架构的底层设施 (网关、服务发现、配置中心、链路追踪) 建设往往需要消耗 30% 以上的开发精力。在 10 周的项目中, 这几乎等同于“自杀”。单体架构让你们能把 ****100% 的火力****集中在核心业务逻辑 (如跑腿匹配算法、积分结算安全) 上。

2. “模块化”是留给未来的后门

你们提到的 ``com.platform.task``, ``com.platform.user`` 包划分非常关键。

> ****架构师建议:**** 即使在同一个数据库中, 也要尽量****避免跨模块的数据库联表查询 (Join)****。如果跑腿模块需要用户信息, 通过定义的内部 Service 接口获取, 而不是直接查询用户表。这样未来如果真的要拆分微服务, 只需要把内部接口调用改为 RPC 调用即可。

🚀 核心功能点的技术实现分析

针对问卷中用户提到的几个痛点，从架构层面我补充几个考量维度：

🛡️ 安全与认证（学号认证）

- * **隐私平衡：** 问卷显示用户既想要学号认证的安全性，又想要匿名的自由度。
- * **方案建议：** 采用 **JWT (JSON Web Token)** 进行状态管理。后端在认证学号后，允许用户设置一个“展示昵称”。数据库存储真实的学号关联，但对外接口仅展示 **UID** 或昵称，实现“实名认证，匿名社交”。

💰 酬劳与中间人机制（Escrow）

用户提到的“平台作为中间人保留酬劳”是典型的**托管协议（Escrow Pattern）**。

- * **流程建议：**
 - 1. 发单者预扣积分/现金 → 2. 平台锁定该笔资金 → 3. 接单者完成任务 → 4. 双方确认 → 5. 平台释放资金。
- * **架构挑战：** 即使是单体架构，也要确保这个过程的**原子性**。必须使用数据库事务（Database Transaction）确保：不能出现“积分扣了，但订单没生成”的情况。

💬 交流与反馈

- * **即时性需求：** 考虑到 10 周时间，不建议从零开发 **WebSocket** 聊天服务器。
- * **方案建议：**
 - * **低配版：** 采用“轮询”或直接接入成熟的第三方 **IM SDK**（如腾讯云 IM、网易云信）。
 - * **折中版：** 仅做任务下方的留言板（类似闲鱼鱼塘），利用推送通知告知用户。

📊 架构演进建议表

为了确保你们的系统不仅能跑起来，还能“活下去”，我建议关注以下演进路径：

阶段	关注重点	技术手段
:---	:---	:---
初期 (W1-W4)	核心业务闭环	快速完成分层架构，定义好数据库 Schema，完成学号认证。
中期 (W5-W8)	交易安全性	实现积分锁定的事务逻辑，开发评价反馈体系。
后期 (W9-W10)	性能与体验	增加 Redis 缓存 （针对高频访问的失物招领），优化搜索筛选。

💡 架构师的最后嘱托

在单体应用中，最容易腐坏的是**数据模型**。

> **警告：** 不要让 `Order` 对象里既包含跑腿信息，又包含二手交易信息，还包含学习资料信息。虽然它们都是“订单”，但业务属性完全不同。建议采用**策略模式**或者**类继承**，让不同的互助服务逻辑解耦。

你们目前的思路非常扎实。如果能在开发过程中坚持 **“高内聚、低耦合”** 的代码约束，这个平台在校赛甚至真实运营中都会有很强的生命力。

对于这套“模块化单体”方案，你目前最担心的是哪一个功能模块的实现逻辑？或者是数据库设计的复杂度？

辩论结论

针对我们的校园互助平台，经过激烈讨论，我们最终选择 **方案 A（分层架构）**。

理由如下：

1. **交付压力：** 10 周的时间非常紧迫。微服务带来的“分布式事务”和“服务间治理”会占据大量开发时间，导致核心业务逻辑（如积分结算、评价机制）无法按时完成。
2. **业务初期规模：** 校园平台的初期并发量通常不会达到必须使用微服务的量级。
3. **折中方案：** 你们可以采用“**模块化单体**”的思路。在方案 A 的基础上，代码层面保持严格的模块化（例如 `com.platform.task`, `com.platform.user`），这样未来如果用户量真的突破万级，再剥离成微服务也会非常轻松。